NASM32 Assembly Programming Laboratory

Four programming problems | Conditional jumps, procedures, validation, and arithmetic

Student: Cayanan, Nathaniel C. Section: TN24 Date: _____

General Instructions

• Develop four separate 32-bit NASM assembly programs. Use the instructor-provided template and the required operating-system conventions.

• Use named procedures (CALL/RET) and conditional/unconditional jumps to implement control flow. Use .data, .bss, and .text sections as appropriate.

• Read full names with spaces. Strip input line endings before displaying text. For numeric inputs, perform ASCII-to-integer conversion and validate malformed input.

• For each problem, submit one .asm source file and screenshots showing the compilation/linking commands and all required test-case outputs.

• Do not submit screenshots alone. Include brief written answers to the analysis questions for each problem.

Compilation example (32-bit Linux, NASM + ld): nasm -f elf32 problem1.asm -o problem1.o | ld -m elf_i386 problem1.o -o problem1 | ./problem1

Note: On other operating systems or when using C-library functions, compilation/linking commands may differ. Use the method demonstrated in class.

Problem 1 — Name, Age, and Gender Greeting

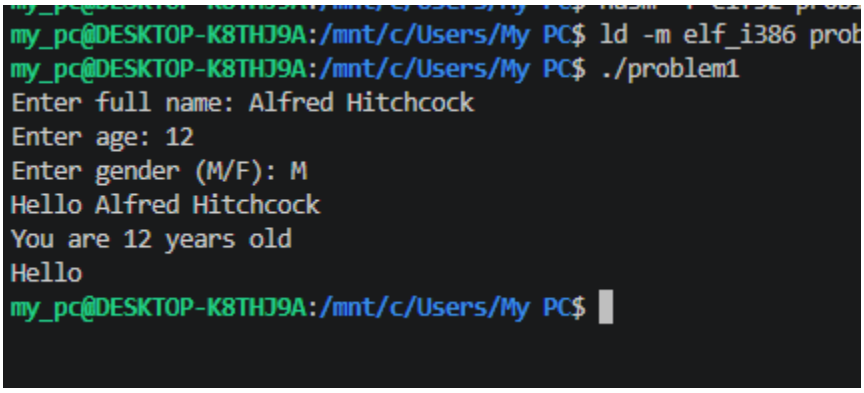
Difficulty: Basic | Objective: Practice input, character comparisons, jumps, and procedures.

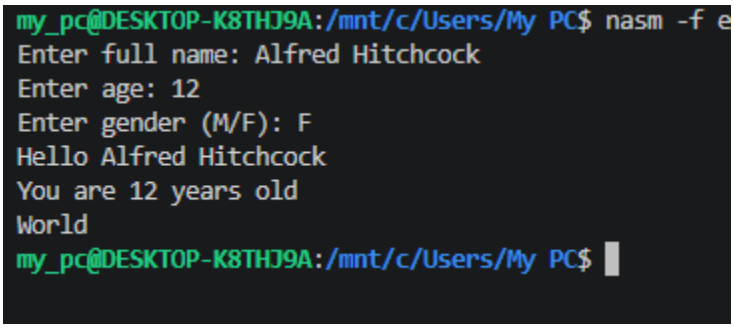
Write a 32-bit NASM program that asks for the user's full name, age, and gender (M or F). Print "Hello <name>" and "You are <age> years old". Then print "Hello" for M, "World" for F, and "INVALID" for any other input. Accept lowercase m/f as equivalent to uppercase (extension).

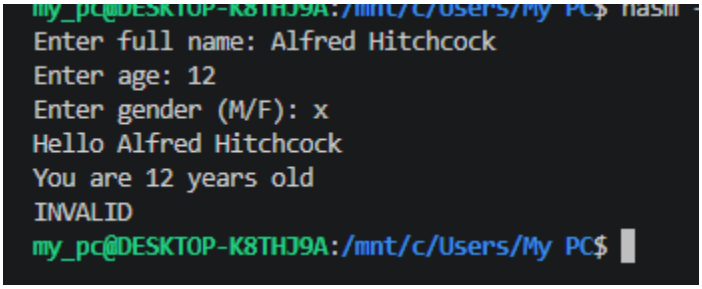
Required test cases

```
What is your name? Alfred Hitchcock
How old are you? 12
What is your gender? M
Hello Alfred Hitchcock
You are 12 years old
Hello
```

Repeat with F (final line: World) and X (final line: INVALID).







Student questions

• Which conditional jump distinguishes M from F?

- CMP then JE aor JZ

• What happens when the gender is neither M nor F?

- It goes to the invalid case.

• Why should the name be stored in a buffer rather than a single byte?

- To prevent data truncation or memory corruption.

Problem 2 — Student Year-Level Classification

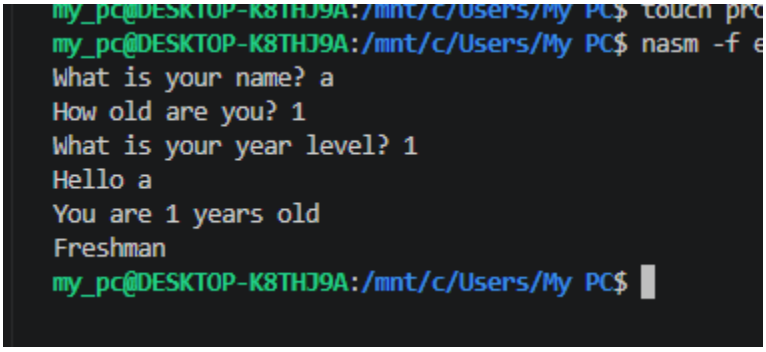
Difficulty: Basic–Intermediate | Objective: Implement a multi-branch selection.

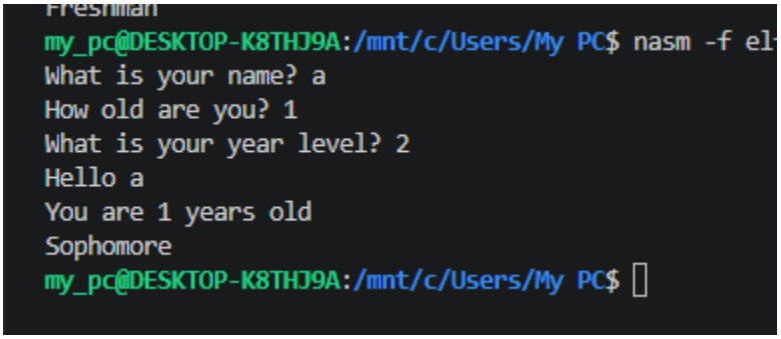
Ask for full name, age, and year level. Display “Hello <name>” and “You are <age> years old”. Classify 1 as Freshman, 2 as Sophomore, 3 as Junior, 4 as Senior; otherwise display “INVALID YEAR LEVEL”.

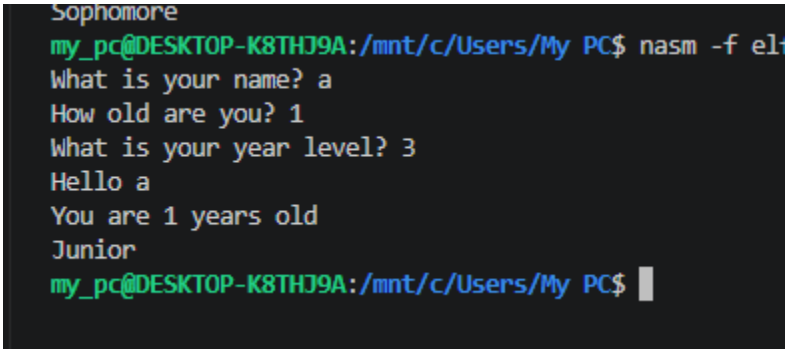
Required test cases

```
What is your name? Juan Dela Cruz
How old are you? 18
What is your year level? 1
Hello Juan Dela Cruz
You are 18 years old
Freshman
```

Also test levels 2, 3, 4, and 5; expected final lines: Sophomore, Junior, Senior, and INVALID YEAR LEVEL, respectively.







```
Junior
my_pc@DESKTOP-K8THJ9A:/mnt/c/Users/My PC$ nasm
What is your name? a
How old are you? 1
What is your year level? 4
Hello a
You are 1 years old
Senior
my_pc@DESKTOP-K8THJ9A:/mnt/c/Users/My PC$
```

Student questions

• How many comparisons are needed in your chosen control-flow design?

- There are four comparisons.

• How do you prevent execution from falling through into another classification?

- From the jmp exit_program into each block.

• How does your program handle an invalid year-level input?

- It falls into a default called case_invalid_year and displays an error.

Problem 3 — Student Grade Evaluation

Difficulty: Intermediate | Objective: Compare numeric ranges and validate input.

Ask for full name, student ID, and integer grade. Display the entered details and the correct classification. Grades outside 0–100 or nonnumeric grades must be rejected. Treat the student ID as text so leading zeros are preserved.

Grade range	Classification
90–100	Excellent
85–89	Very Good
80–84	Good
75–79	Passed
0–74	Failed
Outside 0–100 or nonnumeric	INVALID GRADE

Required test cases

```
Enter student name: Juan Dela Cruz
Enter student ID: 20260001
Enter grade: 95
Student Name: Juan Dela Cruz
Student ID: 20260001
Grade: 95
Classification: Excellent
```

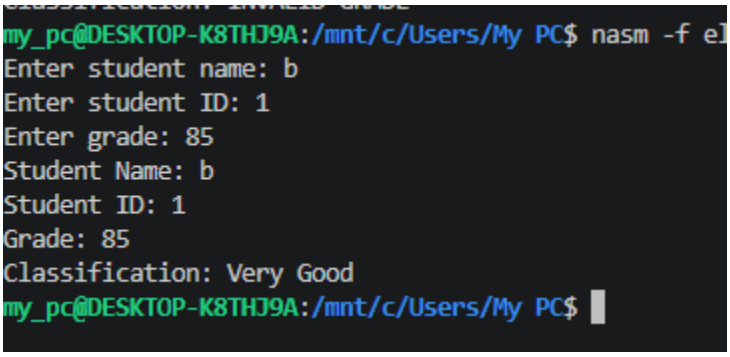
Also test grades 77 (Passed), 68 (Failed), 105 (INVALID GRADE), and 85 (Very Good).

```
my_pc@DESKTOP-K8THJ9A:/mnt/c/Users/My PC$ nasm
Enter student name: b
Enter student ID: 1
Enter grade: 95
Student Name: b
Student ID: 1
Grade: 95
Classification: Excellent
my_pc@DESKTOP-K8THJ9A:/mnt/c/Users/My PC$
```

```
my_pc@DESKTOP-K8THJ9A:/mnt/c/Users/My PC$ na
Enter student name: b
Enter student ID: 1
Enter grade: 77
Student Name: b
Student ID: 1
Grade: 77
Classification: Passed
my_pc@DESKTOP-K8THJ9A:/mnt/c/Users/My PC$
```

```
Grade: 77
Classification: Passed
my_pc@DESKTOP-K8THJ9A:/mnt/c/Users/My PC$ nasm -
Enter student name: b
Enter student ID: 1
Enter grade: 68
Student Name: b
Student ID: 1
Grade: 68
Classification: Failed
my_pc@DESKTOP-K8THJ9A:/mnt/c/Users/My PC$
```

```
Grade: 68
Classification: Failed
my_pc@DESKTOP-K8THJ9A:/mnt/c/Users/My PC$ nasm
Enter student name: v
Enter student ID: 1
Enter grade: 105
Student Name: v
Student ID: 1
Grade: 105
Classification: INVALID GRADE
my_pc@DESKTOP-K8THJ9A:/mnt/c/Users/My PC$
```



Student questions

• Why must ASCII digits be converted before comparing grade ranges?

- For the CPU to process it more easily.

• In what order did you check the grade boundaries, and why?

- I ordered it in a descending order to easily use a greater than or equal logical operator.

• How would you handle a grade of exactly 75 or 90?

- By using the JGE, it automatically goes to pass or excellent.

Problem 4 — ATM Withdrawal Validation

Difficulty: Intermediate–Challenging | Objective: Combine authentication, arithmetic, and nested validation.

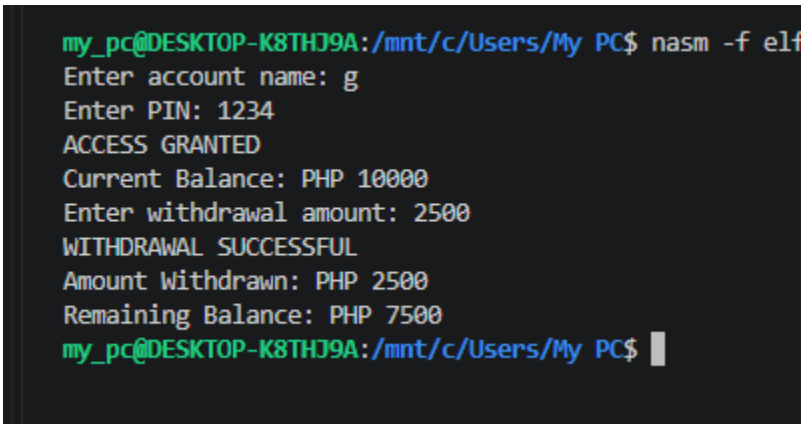
Simulate an ATM with starting balance PHP 10000 and demonstration PIN 1234. Ask for account name and PIN. If PIN is wrong, display ACCESS DENIED and terminate. Otherwise show ACCESS GRANTED and the balance, then request a withdrawal amount. Check in this order: amount must be a positive integer; amount must not exceed balance; amount must be a multiple of 100. Show the corresponding error message or deduct the amount and print the new balance. This is a classroom simulation, not a secure banking implementation.

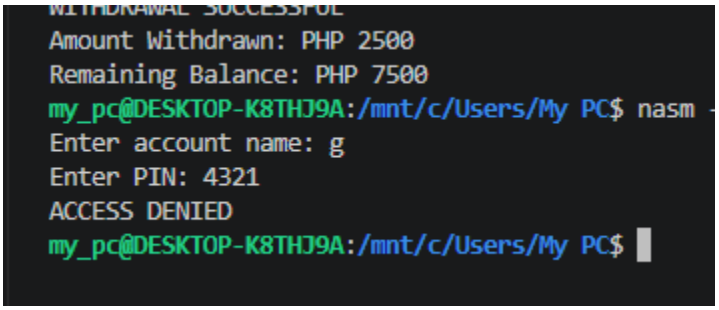
Condition	Output
Incorrect PIN	ACCESS DENIED
Amount ≤ 0 or nonnumeric	INVALID AMOUNT
Amount > balance	INSUFFICIENT FUNDS
Amount not divisible by 100	AMOUNT MUST BE A MULTIPLE OF 100
All checks pass	WITHDRAWAL SUCCESSFUL

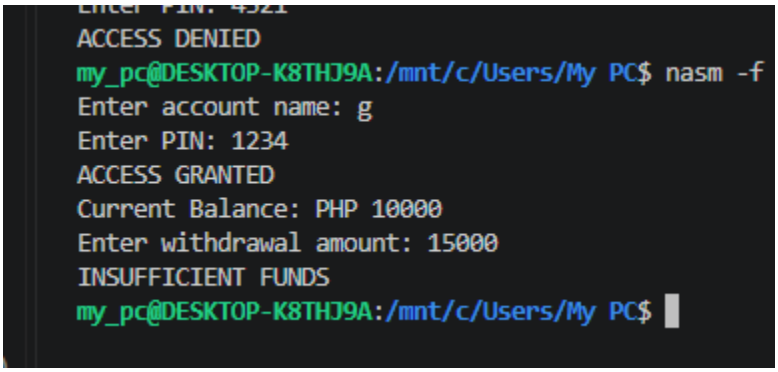
Required test cases

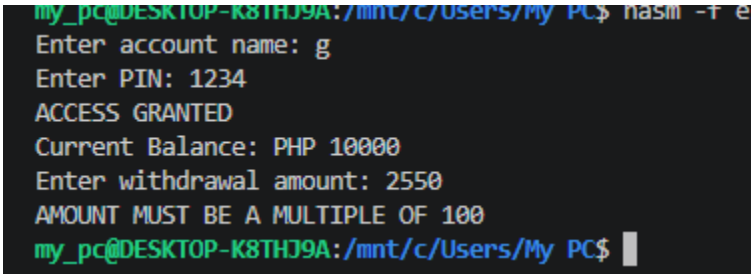
```
Enter account name: Juan Dela Cruz
Enter PIN: 1234
ACCESS GRANTED
Current Balance: PHP 10000
Enter withdrawal amount: 2500
WITHDRAWAL SUCCESSFUL
Amount Withdrawn: PHP 2500
Remaining Balance: PHP 7500
```

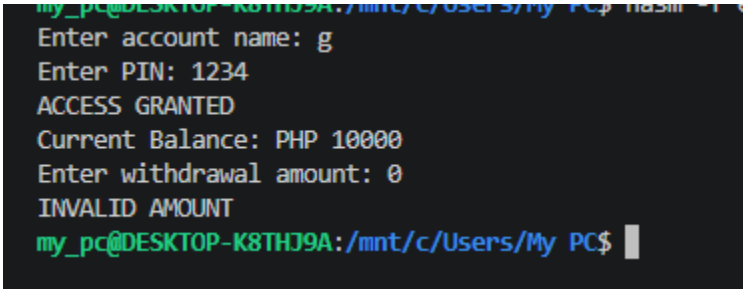
Also test PIN 4321 (ACCESS DENIED), withdrawal 15000 (INSUFFICIENT FUNDS), 2550 (AMOUNT MUST BE A MULTIPLE OF 100), and 0 (INVALID AMOUNT). Use PIN 1234 for withdrawal tests.











Student questions

• Why must PIN verification happen before displaying the account balance?

- Because an ATM is very confidential and you must first verify the PIN to avoid others accessing it.

• Which arithmetic instruction updates the balance after a successful withdrawal?

- Subtract is used to deduct the amount in the ATM.

• How can a remainder be used to test whether an amount is a multiple of 100?

- Because of the divide and will only be divided by 100.

• What happens to the balance after a rejected withdrawal?

- The balance remains the same.

Submission Checklist

☐ Four .asm files, one per problem

☐ Compilation/linking evidence and test-case screenshots for every problem

☐ Answers to all analysis questions

☐ Programs use procedures, jumps, and appropriate input validation

Provide comprehensive Test Cases, include screenshot of each Test case

Suggested filenames: problem1_gender.asm, problem2_yearlevel.asm, problem3_grades.asm, problem4_atm.asm